

Isotope Tracing for Apache Kafka

Lightweight, end-to-end record tracing for Kafka built entirely on **public extension points** — a **ProducerInterceptor** plus record headers. No broker changes, no agent, no vendor lock-in. Stamp a UUIDv7 trace id and origin metadata on first produce, accumulate a hop on every re-produce, and reconstruct latency / topology / hop-distribution from the headers (or from the optional **Prometheus metrics** or **OTLP spans**).

Three artifacts, so you only take what you need:

Module	Coordinate	Pulls	Use it for
kafka-isotope-core	ai.signalroom:kafka-isotope-core	Jackson, SLF4J (kafka-clients is compileOnly)	Trace propagation — the interceptor, headers, consume markers.
kafka-isotope-metrics	ai.signalroom:kafka-isotope-metrics	kafka-isotope-core + Micrometer/Prometheus	Optional /metrics exporter for the stateless reports.
kafka-isotope-otel	ai.signalroom:kafka-isotope-otel	kafka-isotope-core + OTLP protobuf	Optional span writer — one OTLP span per hop on a Kafka topic.

kafka-isotope-core has no metrics or tracing dependency. Emission is routed through the **IsotopeMetricsSink** and **IsotopeSpanSink** interfaces, which start out as no-ops. Adding **kafka-isotope-metrics** or **kafka-isotope-otel** to the classpath doesn't change that: the real sink is registered only when your application calls **PrometheusIsotopeMetrics.start(port)** or **KafkaOtlpSpanSink.start(config)** (steps §2.3 and §2.4 below), and stays a no-op if that call fails. Until then, each record costs an "is it enabled?" check and no metric or span work.

Table of Contents

- 1.0 Install
- 2.0 Put It To Work
 - 2.1 Produce Side — Register the Interceptor
 - 2.2 Consume Side — Adopt to Continue the Trace, or Mark a Terminal Consume
 - 2.3 [OPTIONAL] Metrics — Start the Prometheus Exporter Once at Boot
 - 2.4 [OPTIONAL] OTel Spans — Start the OpenTelemetry Protocol (OTLP) Span Writer Once at Boot
- 3.0 Where to Find the Code

1.0 Install

```

repositories {
    mavenCentral()
}
dependencies {
    implementation 'ai.signalroom:kafka-isotope-core:0.19.0'
    implementation 'ai.signalroom:kafka-isotope-metrics:0.19.0' // optional
    - only for Prometheus metrics
    implementation 'ai.signalroom:kafka-isotope-otel:0.19.0' // optional
    - only for OTel spans
}

```

2.0 Put It To Work

Steps 2.1 and 2.2 are required; 2.3 and 2.4 are optional.

2.1 Produce Side — Register the Interceptor

Every `send()` is stamped/hopped automatically; nothing else to call.

```

props.put(ProducerConfig.INTERCEPTOR_CLASSES_CONFIG,
           IsotopeProducerInterceptor.class.getName());
props.put(IsotopeProducerInterceptor.SERVICE_NAME_CONFIG, "order-intake-
service");
props.put(IsotopeProducerInterceptor.PIPELINE_NAME_CONFIG, "orders");

```

2.2 Consume Side — Adopt to Continue the Trace, or Mark a Terminal Consume

Do one of these for each record you consume. A stage that re-produces adopts the trace, so its next `send()` becomes the next hop. A stage where the pipeline ends writes a consume marker instead.

```

// A stage that consumes then re-produces ("hop"): adopt so the next send()
// continues the same trace instead of starting a new one.
IsotopeContext.adaptFromRecord(record, "order-enrichment-service");
// ... process and produce downstream; the interceptor reads the adopted
context ...
IsotopeContext.clear(); // per-record, on the same thread

// A terminal consumer (no re-produce): write a bipartite consume-edge
marker.
IsotopeContext.recordConsume(record, "shipping-notification-service",
markerProducer);

```

`markerProducer` is a `Producer<byte[], byte[]>` you create just for markers, and it must **not** have `IsotopeProducerInterceptor` in its `interceptor.classes`. If it does, the interceptor treats each marker as a produce of its own and overwrites the trace headers the marker exists to forward. Markers go to `isotope_consume_edge_markers` by default, and closing the producer is up to you.

2.3 [OPTIONAL] Metrics — Start the Prometheus Exporter Once at Boot

```
import ai.signalroom.kafka.isotope.metrics.PrometheusIsotopeMetrics;

PrometheusIsotopeMetrics.start(9404); // serves GET /metrics; no-op on a
port clash
```

This serves `isotope_hop_latency_*`, `isotope_hop_records_total`, and the consume-side metrics at `http://localhost:9404/metrics`. Until you call `start`, the interceptor's metric calls are inert. See [kafka-isotope-metrics/README.md](#).

2.4 [OPTIONAL] OTEL Spans — Start the OpenTelemetry Protocol (OTLP) Span Writer Once at Boot

```
import ai.signalroom.kafka.isotope.otel.KafkaOtlpSpanSink;

KafkaOtlpSpanSink.start(Map.of(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG,
" kafka:9092"));

// ... at shutdown:
KafkaOtlpSpanSink.close();
```

Every hop then lands on `isotope_trace_spans` as an [OTLP span](#), ready for a stock OpenTelemetry Collector. Queued off the hot path and dropped rather than allowed to delay a `send()`. At shutdown, `close()` releases the writer; its producer flushes any queued spans and closes once your producers (and their interceptors) have closed too. See [kafka-isotope-otel/README.md](#).

3.0 Where to Find the Code

- `Isotope` — the header model + JSON codec, UUIDv7, `MAX_HOPS`.
- `IsotopeContext` — thread-local context, `adoptFromRecord`, `recordConsume`.
- `IsotopeProducerInterceptor` — the `ProducerInterceptor` that stamps and hops.
- `IsotopeMetrics` / `IsotopeMetricsSink` — the metrics seam (core stays metrics-free).
- `IsotopeSpans` / `IsotopeSpanSink` — the span seam (core stays tracing-free).

For the full metrics and PromQL reference and the design rationale (why only three of seven reports are metrics-native), see [docs/metrics.md](#) and [docs/design.md](#). The runnable demo and one-command Prometheus + Grafana showcase live in the `app` module and [k8s/monitoring](#).